

Name: Hana Samy Ahmed
Group:3

SV Project

Design Before:

```
////////////////////////////////////  
// Author: Kareem Waseem  
// Course: Digital Verification using SV & UVM  
//  
// Description: FIFO Design  
//  
////////////////////////////////////  
module FIFO(data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull,  
almostempty, wr_ack, overflow, underflow, data_out);  
parameter FIFO_WIDTH = 16;  
parameter FIFO_DEPTH = 8;  
input [FIFO_WIDTH-1:0] data_in;  
input clk, rst_n, wr_en, rd_en;  
output reg [FIFO_WIDTH-1:0] data_out;  
output reg wr_ack, overflow;  
output full, empty, almostfull, almostempty, underflow;  
  
localparam max_fifo_addr = $clog2(FIFO_DEPTH);  
  
reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];  
  
reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;  
reg [max_fifo_addr:0] count;  
  
always @(posedge clk or negedge rst_n) begin  
    if (!rst_n) begin  
        wr_ptr <= 0;  
    end  
    else if (wr_en && count < FIFO_DEPTH) begin  
        mem[wr_ptr] <= data_in;  
        wr_ack <= 1;  
        wr_ptr <= wr_ptr + 1;  
    end  
    else begin
```

```

        wr_ack <= 0;
        if (full & wr_en)
            overflow <= 1;
        else
            overflow <= 0;
        end
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0;
    end
    else if (rd_en && count != 0) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({wr_en, rd_en} == 2'b10) && !full)
            count <= count + 1;
        else if ( ({wr_en, rd_en} == 2'b01) && !empty)
            count <= count - 1;
    end
end

assign full = (count == FIFO_DEPTH)? 1 : 0;
assign empty = (count == 0)? 1 : 0;
assign underflow = (empty && rd_en)? 1 : 0;
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
assign almostempty = (count == 1)? 1 : 0;

endmodule

```

Design Bugs:

1. Reset signals overflow, wr_ack and underflow (data_out not to be included)
2. Unhandled 2 cases:

-If a read and write enables were high and the FIFO was empty, only writing will take place.

-If a read and write enables were high and the FIFO was full, only reading will take place.

3. underflow is sequential not combinational

4. almostfull flag: FIFO_DEPTH-2 corrected to FIFO_DEPTH-1

Verification Plan:

1	Label	Description	Stimulus Generation	Functional Coverage	Functionality Check
2	LABEL1	When the rst_n is asserted, All flags & internal signals should equal 0	Randomized less often 95 off & 5% on during the simulation		Immediate assertion to check async rst_n functionality as it is async
3	LABEL2	When rst_n is deactivated & the FIFO is full of elements (conut=depth). The full flag should be high	Randomized, Write enable to be high with distribution of the value WR_EN_ON_DIST and to be low with 100-WR_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_FULL_CROSS, covers write & read enables and full flag	Immediate assertion to check full flag functionality as it is combinational
4	LABEL3	When rst_n is deactivated & the FIFO has one element left (conut=depth-1). The almostfull flag should be high	Randomized, Write enable to be high with distribution of the value WR_EN_ON_DIST and to be low with 100-WR_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_ALMOST_FULL_CROSS, covers write & read enables and almostfull flag	Immediate assertion to check almostfull flag functionality as it is combinational
5	LABEL4	When rst_n is deactivated & the FIFO has no element inside (conut=0). The empty flag should be high	Randomized, Read enable to be high with distribution of the value RD_EN_ON_DIST and to be low with 100-RD_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_EMPTY_CROSS, covers write & read enables and empty flag	Immediate assertion to check empty flag functionality as it is combinational
6	LABEL5	When rst_n is deactivated & the FIFO has only one element inside (conut=1). The empty flag should be high	Randomized, Read enable to be high with distribution of the value RD_EN_ON_DIST and to be low with 100-RD_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_ALMSOT_EMPTY_CROSS, covers write & read enables and almostempty flag	Immediate assertion to check almostempty flag functionality as it is combinational
7	LABEL6	When rst_n is deactivated, You want to read & the FIFO is not full, The wr_ack flag should be high and wr_ptr should increment	Randomized, Write enable to be high with distribution of the value WR_EN_ON_DIST and to be low with 100-WR_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_WR_ACK_CROSS, covers write & read enables and wr_ack flag	Concurrent assertion to check wr_ptr & wr_ack flag functionality as they are sequential
8	LABEL7	When rst_n is deactivated, You want to read & the FIFO is full, The overflow flag should be high	Randomized, Write enable to be high with distribution of the value WR_EN_ON_DIST and to be low with 100-WR_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_OVERFLOW_CROSS, covers write & read enables and overflow flag	Concurrent assertion to check overflow flag functionality as it is sequential
9	LABEL8	When rst_n is deactivated, You want to read & the FIFO is not empty rd_ptr should increment and data_out should equal mem[rd_ptr]	Randomized, Read enable to be high with distribution of the value RD_EN_ON_DIST and to be low with 100-RD_EN_ON_DIST during the simulation		Concurrent assertion to check rd_ptr functionality as it is sequential, data_out checked against reference model to check functionality
10	LABEL9	When rst_n is deactivated, You want to read & the FIFO is empty underflow flag should be high	Randomized, Read enable to be high with distribution of the value RD_EN_ON_DIST and to be low with 100-RD_EN_ON_DIST during the simulation	Cross coverage: WRITE_READ_UNDERFLOW_CROSS, covers write & read enables and underflow flag	Concurrent assertion to check underflow flag functionality as it is sequential

Design After (with Assertions):

```

////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
// Description: FIFO Design
////////////////////////////////////
module FIFO (FIFO_INTERFACE.DUT fif);

reg [fif.max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [fif.max_fifo_addr:0] count;

reg [fif.FIFO_WIDTH-1:0] mem [fif.FIFO_DEPTH-1:0];

```

```

always @(posedge fif.clk or negedge fif.rst_n) begin
    if (!fif.rst_n) begin
        wr_ptr <= 0;
        // Bug detected: Reset signals fif.overflow & fif.wr_ack
        fif.overflow <= 0;
        fif.wr_ack <= 0;
    end
    else if (fif.wr_en && count < fif.FIFO_DEPTH) begin
        mem[wr_ptr] <= fif.data_in;
        fif.wr_ack<=1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        fif.wr_ack <= 0;
        if (fif.full & fif.wr_en)
            fif.overflow <= 1;
        else
            fif.overflow <= 0;
    end
end

always @(posedge fif.clk or negedge fif.rst_n) begin
    if (!fif.rst_n) begin
        rd_ptr <= 0;
        // Bug detected: Reset signals fif.underflow
        fif.underflow <= 0;
    end
    else if (fif.rd_en && count != 0) begin
        fif.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
    // Handled fif.underflow behaviour , turned from combinational to sequential
    else begin
        if (fif.empty & fif.rd_en)
            fif.underflow <= 1;
        else
            fif.underflow <= 0;
    end
end

always @(posedge fif.clk or negedge fif.rst_n) begin
    if (!fif.rst_n) begin
        count <= 0;
    end
end

```

```

    else begin
        if ( ({fif.wr_en, fif.rd_en} == 2'b10) && !fif.full)
            count <= count + 1;
        else if ( ({fif.wr_en, fif.rd_en} == 2'b01) && !fif.empty)
            count <= count - 1;
        // Bug detected: Unhandled case, If a read and write enables were high
        and the FIFO was fif.empty, only writing will take place.
        else if ( ({fif.wr_en, fif.rd_en} == 2'b11) && fif.empty)
            count <= count + 1;
        // Bug detected: Unhandled cases, If a read and write enables were high
        and the FIFO was fif.full, only reading will take place.
        else if ( ({fif.wr_en, fif.rd_en} == 2'b11) && fif.full)
            count <= count - 1;
    end
end

assign fif.full = (count == fif.FIFO_DEPTH)? 1 : 0;
assign fif.empty = (count == 0)? 1 : 0;
assign fif.almostfull = (count == fif.FIFO_DEPTH-1)? 1 : 0; // Bug detected:
fif.FIFO_DEPTH-2 --> fif.FIFO_DEPTH-1
assign fif.almostempty = (count == 1)? 1 : 0;

// Guarded assertions
`ifdef SIM
    // Properties, Assertions & Covers
    always_comb begin
        if(!fif.rst_n)
            reset_1_assertion: assert final
((!fif.wr_ack)&&(!fif.overflow)&&(!fif.underflow)&&(!wr_ptr)&&(!rd_ptr)&&(!count)
);
            reset_1_cover: cover final
((!fif.wr_ack)&&(!fif.overflow)&&(!fif.underflow)&&(!wr_ptr)&&(!rd_ptr)&&(!count)
);
    end

    always_comb begin
        if((fif.rst_n)&&(count == fif.FIFO_DEPTH))
            full_assertion: assert final (fif.full);
            full_cover: cover (fif.full);
    end

    always_comb begin
        if((fif.rst_n)&&(count == 0))
            empty_assertion: assert final (fif.empty);
            empty_cover: cover (fif.empty);
    end

```

```

end

always_comb begin
    if((fif.rst_n)&&(count == fif.FIFO_DEPTH-1))
        almostfull_assertion: assert final (fif.almostfull);
        almostfull_cover: cover (fif.almostfull);
    end

always_comb begin
    if((fif.rst_n)&&(count == 1))
        almostempty_assertion: assert final (fif.almostempty);
        almostempty_cover: cover (fif.almostempty);
    end

property P1;
    @(posedge fif.clk or negedge fif.rst_n)
        (!fif.rst_n) |->
((!fif.wr_ack)&&(!fif.overflow)&&(!fif.underflow)&&(!wr_ptr)&&(!rd_ptr)&&(!count)
);
    endproperty

property P2;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        (fif.wr_en && !fif.full ) |=>
((fif.wr_ack)&&((wr_ptr==$past(wr_ptr)+1)|| (wr_ptr==0 && $past(wr_ptr) +1 ==
8)));
    endproperty

property P3;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        (fif.full & fif.wr_en) |=> (fif.overflow);
    endproperty

property P4;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        (fif.rd_en && count != 0) |=> ((rd_ptr==$past(rd_ptr)+1)|| (rd_ptr==0 &&
$past(rd_ptr) +1 == 8));
    endproperty

property P5;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        (fif.empty & fif.rd_en) |=> (fif.underflow);
    endproperty

```

```

property P6;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        ({fif.wr_en, fif.rd_en} == 2'b10) && !fif.full)   | =>
((count==$past(count)+1)|| (count==0 && $past(count) +1 == 9));
endproperty

property P7;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        ( ({fif.wr_en, fif.rd_en} == 2'b01) && !fif.empty)   | =>
((count==$past(count)-1));
endproperty

property P8;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        ( ({fif.wr_en, fif.rd_en} == 2'b11) && fif.empty)   | =>
((count==$past(count)+1)|| (count==0 && $past(count) +1 == 9));
endproperty

property P9;
    @(posedge fif.clk) disable iff(!fif.rst_n)
        ( ({fif.wr_en, fif.rd_en} == 2'b11) && fif.full)   | =>
((count==$past(count)-1));
endproperty

reset_2_assertion: assert property(P1);
reset_2_cover: cover property(P1);

write_assertion: assert property(P2);
write_cover: cover property(P2);

overflow_assertion: assert property(P3);
overflow_cover: cover property(P3);

read_assertion: assert property(P4);
read_cover: cover property(P4);

underflow_assertion: assert property(P5);
underflow_cover: cover property(P5);

write_not_full_assertion: assert property(P6);
write_not_full_cover: cover property(P6);

read_not_empty_assertion: assert property(P7);
read_not_empty_cover: cover property(P7);

```

```

    read_write_empty_assertion: assert property(P8);
    read_write_empty_cover: cover property(P8);

    read_write_full_assertion: assert property(P9);
    read_write_full_cover: cover property(P9);
`endif
endmodule

```

Shared Package:

```

package shared_pkg;

    logic test_finished;

    // Counters
    integer error_count=0;
    integer correct_count=0;

endpackage : shared_pkg

```

Interface:

```

interface FIFO_INTERFACE (clk);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
localparam max_fifo_addr = $clog2(FIFO_DEPTH);

input bit clk;

logic [FIFO_WIDTH-1:0] data_in;
logic rst_n, wr_en, rd_en;
logic [FIFO_WIDTH-1:0] data_out;
logic wr_ack, overflow;
logic full, empty, almostfull, almostempty, underflow;

modport DUT (input clk, data_in, rst_n, wr_en, rd_en,
             output
data_out, wr_ack, overflow, full, empty, almostempty, almostfull, underflow

);

modport TEST (input
clk, data_out, wr_ack, overflow, full, empty, almostempty, almostfull, underflow,
             output data_in, rst_n, wr_en, rd_en

```



```

);

modport MONITOR (input clk, data_in,rst_n,wr_en,rd_en,
                 data_out,wr_ack,overflow,full,empty,almostempty,almostfull,underflow);

);

endinterface

```

Monitor:

```

import FIFO_coverage::*;
import FIFO_scoreboard::*;
import shared_pkg::*;
import FIFO_transaction::*;

module FIFO_MONITOR(FIFO_INTERFACE.MONITOR fif);

    FIFO_transaction F_trans = new();
    FIFO_scoreboard F_sb = new();
    FIFO_coverage F_cov = new();

    initial begin
        forever begin
            @(negedge fif.clk);
            F_trans.data_in = fif.data_in;
            F_trans.rst_n = fif.rst_n;
            F_trans.wr_en = fif.wr_en;
            F_trans.rd_en = fif.rd_en;
            F_trans.data_out = fif.data_out;
            F_trans.wr_ack= fif.wr_ack;
            F_trans.overflow= fif.overflow;
            F_trans.full= fif.full;
            F_trans.empty= fif.empty;
            F_trans.almostempty= fif.almostempty;
            F_trans.almostfull= fif.almostfull;
            F_trans.underflow= fif.underflow;

            fork
                begin
                    F_cov.sample_data(F_trans);

```

```

        end
        begin
            F_sb.check_data(F_trans);
        end
    join

    if (test_finished)begin
        $display("Test Finished ...correct counts = %0d , Error counts = %0d",correct_count,error_count);
        $stop;
    end
end
end
endmodule

```

Testbench:

```

import shared_pkg::*;
import FIFO_transaction::*;
import FIFO_coverage::*;
import FIFO_scoreboard::*;

module FIFO_tb (FIFO_INTERFACE.TEST fif);

    FIFO_transaction trans_obj_tb = new();
    initial begin

        fif.rst_n=0; fif.rd_en=0; fif.wr_en=0; fif.data_in=0;
        @(negedge fif.clk);
        fif.rst_n=1;
        @(negedge fif.clk);

        repeat(10_000) begin
            assert(trans_obj_tb.randomize());
            fif.rst_n=trans_obj_tb.rst_n;
            fif.rd_en=trans_obj_tb.rd_en;
            fif.wr_en=trans_obj_tb.wr_en;
            fif.data_in=trans_obj_tb.data_in;
            @(negedge fif.clk);
        end

        test_finished =1;
    end
end

```

```
endmodule : FIFO_tb
```

Top:

```
module top ();

    bit clk;

    initial begin
        clk=0;
        forever
            #1 clk = ~clk;
    end

    FIFO_INTERFACE fif (clk);

    FIFO FIFO_DUT (fif);

    FIFO_tb FIFO_TB (fif);

    FIFO_MONITOR FIFO_MON (fif);

endmodule
```

Transaction:

```
package FIFO_transaction;

class FIFO_transaction;

    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    localparam max_fifo_addr = $clog2(FIFO_DEPTH);

    rand logic [FIFO_WIDTH-1:0] data_in;
    rand logic rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;
```

```

integer RD_EN_ON_DIST, WR_EN_ON_DIST;

function new(integer RD_EN_ON_DIST = 30 , integer WR_EN_ON_DIST = 70);
    this.RD_EN_ON_DIST = RD_EN_ON_DIST;
    this.WR_EN_ON_DIST = WR_EN_ON_DIST;
endfunction

constraint rst_cons{
    rst_n dist{0:=5 , 1:=95};
}

constraint wr_cons{
    wr_en dist{0:=100 - WR_EN_ON_DIST , 1:=WR_EN_ON_DIST};
}

constraint rd_cons{
    rd_en dist{0:=100 - RD_EN_ON_DIST , 1:=RD_EN_ON_DIST};
}

endclass
endpackage

```

Coverage:

```

package FIFO_coverage;
import FIFO_transaction::*;
class FIFO_coverage;

FIFO_transaction F_cvg_txn = new();

covergroup cg;

wr_en_cp:coverpoint F_cvg_txn.wr_en;
rd_en_cp:coverpoint F_cvg_txn.rd_en;
wr_ack_cp:coverpoint F_cvg_txn.wr_ack;
overflow_cp:coverpoint F_cvg_txn.overflow;
full_cp:coverpoint F_cvg_txn.full;
empty_cp:coverpoint F_cvg_txn.empty;
almostfull_cp:coverpoint F_cvg_txn.almostfull;
almostempty_cp:coverpoint F_cvg_txn.almostempty;
underflow_cp:coverpoint F_cvg_txn.underflow;

```

```

wr_rd_wrack: cross wr_ack_cp, wr_en_cp,rd_en_cp;
wr_rd_overflow: cross overflow_cp, wr_en_cp,rd_en_cp;
wr_rd_underflow: cross underflow_cp, wr_en_cp,rd_en_cp;
wr_rd_full: cross full_cp, wr_en_cp,rd_en_cp;
wr_rd_empty: cross empty_cp, wr_en_cp,rd_en_cp;
wr_rd_almostempty: cross almostempty_cp, wr_en_cp,rd_en_cp;
wr_rd_almostfull: cross almostfull_cp, wr_en_cp,rd_en_cp;

endgroup

function new();
cg =new();

endfunction

function void sample_data(input FIFO_transaction F_txn);
F_cvg_txn = F_txn;
cg.sample();
endfunction

endclass
endpackage

```

Scoreboard:

```

package FIFO_scoreboard;
import FIFO_transaction_pkg::*;
import shared_pkg::*;

FIFO_transaction FIFO_transaction_object = new();

class FIFO_scoreboard;
// Reference FIFO
bit [FIFO_transaction_object.FIFO_WIDTH-1:0] fifo_queue [$];
int fifo_count = 0;

// Reference Signals
logic [FIFO_transaction_object.FIFO_WIDTH-1:0] data_out_ref;

// check_data Function
function void check_data(input FIFO_transaction obj_one);

reference_model(obj_one);

```

```

        // Compare the data_out with the reference
        if (obj_one.data_out !== data_out_ref) begin
            $display("Error!!, At time %t, data_out %d doesn't equal
data_out_ref %d !!", $time, obj_one.data_out, data_out_ref);
            error_counter++;
        end
        else begin
            $display("Success, At time %t, data_out= %d equals data_out_ref=
%d", $time, obj_one.data_out, data_out_ref);
            correct_counter++;
        end
    endfunction

// reference_model Function
function void reference_model(input FIFO_transaction F_txn);
    // Reset logic
    if (!F_txn.rst_n) begin
        fifo_queue <= {};
        fifo_count <= 0;
    end
    else begin
        // Write operation if not full
        if (F_txn.wr_en && fifo_count <
FIFO_transaction_object.FIFO_DEPTH) begin
            fifo_queue.push_back(F_txn.data_in);
            fifo_count <= fifo_queue.size();
        end

        // Read operation if not empty
        if (F_txn.rd_en && fifo_count != 0) begin
            data_out_ref <= fifo_queue.pop_front();
            fifo_count <= fifo_queue.size();
        end
    end

end
endfunction

endclass
endpackage

```

Do file:

 run.do - Notepad

File Edit Format View Help

vlib work

vlog -f FIFO.list +cover -covercells +define+SIM

vsim -voptargs=+acc work.top -cover

add wave *

coverage save FIFO.ucdb -onexit

run -all

#quit -sim

#vcover report FIFO.ucdb -details -annotate -all -output -FIFO_covertr.txt

Source list:

 FIFO.list - Notepad

File Edit Format View Help

shared_pkg.sv

FIFO_transaction.sv

FIFO_coverage.sv

FIFO_scoreboard.sv

Top.sv

FIFO_IF.sv

FIFO.sv

FIFO_tb.sv

FIFO_monitor.sv

Code Coverage:

=== Instance: /top/fif

=== Design Unit: work.FIFO_INTERFACE

=====

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	86	86	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /top/fif --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
almostempty	1	1	100.00
almostfull	1	1	100.00
clk	1	1	100.00
data_in[15-0]	1	1	100.00
data_out[15-0]	1	1	100.00
empty	1	1	100.00
full	1	1	100.00
overflow	1	1	100.00
rd_en	1	1	100.00
rst_n	1	1	100.00
underflow	1	1	100.00
wr_ack	1	1	100.00
wr_en	1	1	100.00

Total Node Count = 43

Toggled Node Count = 43

Untoggled Node Count = 0

Toggle Coverage = 100.00% (86 of 86 bins)

=====


```

Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              35      35        0    100.00%

```

=====Branch Details=====

Branch Coverage for instance /top/FIFO_DUT

```

  Line      Item              Count    Source
  ----      -
File FIFO.sv
-----IF Branch-----
  15              10493    Count coming in to IF
  15              1012      if (!fif.rst_n) begin
  21              5007      else if (fif.wr_en && count < fif.FIFO_DEPTH)
  26              4474      else begin

```

Branch totals: 3 hits of 3 branches = 100.00%

```

-----IF Branch-----
  28              4474    Count coming in to IF
  28              1647      if (fif.full & fif.wr_en)
  30              2827      else

```

Branch totals: 2 hits of 2 branches = 100.00%

```

-----IF Branch-----
  36              10493    Count coming in to IF
  36              1012      if (!fif.rst_n) begin
  41              2507      else if (fif.rd_en && count != 0) begin
  46              6974      else begin

```

Branch totals: 3 hits of 3 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	32	32	0	100.00%

=====Statement Details=====

Statement Coverage for instance /top/FIFO_DUT --

Line	Item	Count	Source
----	----	-----	-----
File FIFO.sv			
7			module FIFO (FIFO_INTERFACE.DUT fif);
8			
9			reg [fif.max_fifo_addr-1:0] wr_ptr, r
10			reg [fif.max_fifo_addr:0] count;
11			
12			reg [fif.FIFO_WIDTH-1:0] mem [fif.FIF
13			
14	1	10493	always @(posedge fif.clk or negedge f
15			if (!fif.rst_n) begin
16	1	1012	wr_ptr <= 0;
17			// Bug detected: Reset sig
18	1	1012	fif.overflow <= 0;
19	1	1012	fif.wr_ack <= 0;
20			end
21			else if (fif.wr_en && count < fif.l

Assertion Coverage:

```
=====
=== Instance: /top/FIFO_DUT
=== Design Unit: work.FIFO
=====
```

Assertion Coverage:

Assertions	14	14	0	100.00%
------------	----	----	---	---------

Name	File(Line)	Failure Count	Pass Count
/top/FIFO_DUT/reset_1_assertion	FIFO.sv(82)	0	1
/top/FIFO_DUT/full_assertion	FIFO.sv(88)	0	1
/top/FIFO_DUT/empty_assertion	FIFO.sv(94)	0	1
/top/FIFO_DUT/almostfull_assertion	FIFO.sv(100)	0	1
/top/FIFO_DUT/almostempty_assertion	FIFO.sv(106)	0	1
/top/FIFO_DUT/reset_2_assertion	FIFO.sv(156)	0	1
/top/FIFO_DUT/write_assertion	FIFO.sv(159)	0	1
/top/FIFO_DUT/overflow_assertion	FIFO.sv(162)	0	1
/top/FIFO_DUT/read_assertion	FIFO.sv(165)	0	1
/top/FIFO_DUT/underflow_assertion	FIFO.sv(168)	0	1
/top/FIFO_DUT/write_not_full_assertion	FIFO.sv(171)	0	1
/top/FIFO_DUT/read_not_empty_assertion	FIFO.sv(174)	0	1
/top/FIFO_DUT/read_write_empty_assertion	FIFO.sv(177)	0	1
/top/FIFO_DUT/read_write_full_assertion	FIFO.sv(180)	0	1

Coverage Report by instance with details

=====
=== Instance: /top/FIFO_DUT
=== Design Unit: work.FIFO
=====

Directive Coverage:
Directives 14 14 0 100.00%

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/top/FIFO_DUT/reset_1_cover	FIFO	Verilog	SVA	FIFO.sv(83)	1002	Covered
/top/FIFO_DUT/full_cover	FIFO	Verilog	SVA	FIFO.sv(89)	923	Covered
/top/FIFO_DUT/empty_cover	FIFO	Verilog	SVA	FIFO.sv(95)	1057	Covered
/top/FIFO_DUT/almostfull_cover	FIFO	Verilog	SVA	FIFO.sv(101)	1111	Covered
/top/FIFO_DUT/almostempty_cover	FIFO	Verilog	SVA	FIFO.sv(107)	675	Covered
/top/FIFO_DUT/reset_2_cover	FIFO	Verilog	SVA	FIFO.sv(157)	521	Covered
/top/FIFO_DUT/write_cover	FIFO	Verilog	SVA	FIFO.sv(160)	4748	Covered
/top/FIFO_DUT/overflow_cover	FIFO	Verilog	SVA	FIFO.sv(163)	1560	Covered
/top/FIFO_DUT/read_cover	FIFO	Verilog	SVA	FIFO.sv(166)	2386	Covered
/top/FIFO_DUT/underflow_cover	FIFO	Verilog	SVA	FIFO.sv(169)	234	Covered
/top/FIFO_DUT/write_not_full_cover	FIFO	Verilog	SVA	FIFO.sv(172)	3339	Covered
/top/FIFO_DUT/read_not_empty_cover	FIFO	Verilog	SVA	FIFO.sv(175)	695	Covered
/top/FIFO_DUT/read_write_empty_cover	FIFO	Verilog	SVA	FIFO.sv(178)	153	Covered
/top/FIFO_DUT/read_write_full_cover	FIFO	Verilog	SVA	FIFO.sv(181)	435	Covered

Functional Coverage:

=====

=== Instance: /FIFO_coverage

=== Design Unit: work.FIFO_coverage

=====

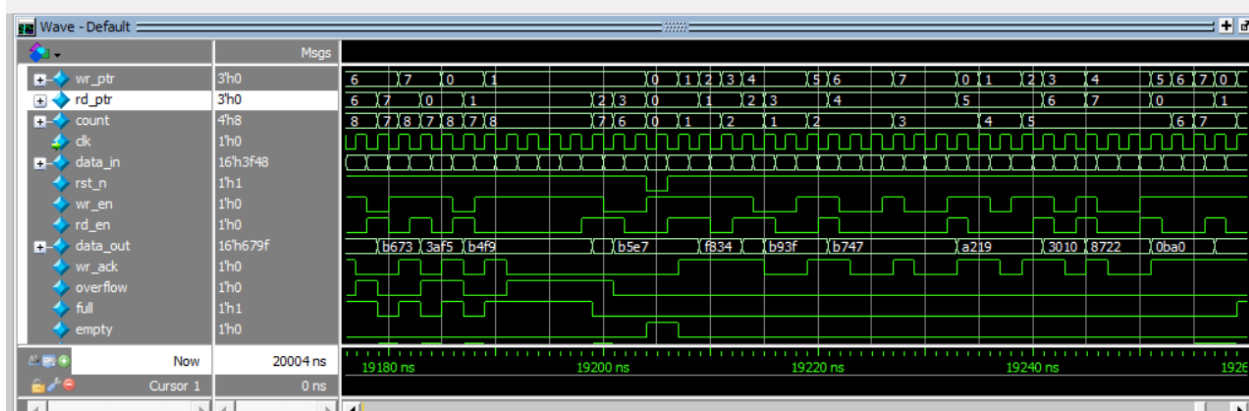
Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	16	na	na	na
Covergroup Bins	74	74	0	100.00%

Covergroup	Metric	Goal	Bins	Status

TYPE /FIFO_coverage/FIFO_coverage/cg	100.00%	100	-	Covered
covered/total bins:	74	74	-	
missing/total bins:	0	74	-	
% Hit:	100.00%	100	-	
Coverpoint wr_en_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	2985	1	-	Covered
bin auto[1]	7017	1	-	Covered
Coverpoint rd_en_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	7097	1	-	Covered
bin auto[1]	2905	1	-	Covered
Coverpoint wr_ack_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	4995	1	-	Covered
bin auto[1]	5007	1	-	Covered
Coverpoint overflow_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	7991	1	-	Covered
bin auto[1]	2011	1	-	Covered
Coverpoint full_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	7517	1	-	Covered

Waveform Snippets:



```
# Success, At time      19986, data_out= 46929 equals data_out_ref= 46929
# Success, At time      19988, data_out= 46929 equals data_out_ref= 46929
# Success, At time      19990, data_out= 59282 equals data_out_ref= 59282
# Success, At time      19992, data_out= 32547 equals data_out_ref= 32547
# Success, At time      19994, data_out= 50802 equals data_out_ref= 50802
# Success, At time      19996, data_out= 26527 equals data_out_ref= 26527
# Success, At time      19998, data_out= 26527 equals data_out_ref= 26527
# Success, At time      20000, data_out= 26527 equals data_out_ref= 26527
# Success, At time      20002, data_out= 26527 equals data_out_ref= 26527
# Success, At time      20004, data_out= 26527 equals data_out_ref= 26527
# Test Finished ...correct counts = 10002 , Error counts = 0
# ** Note: $stop      : FIFO_monitor.sv(39)
#   Time: 20004 ns  Iteration: 1  Instance: /top/FIFO_MON
```